অধ্যায় ১৯

JavaScript Performance Optimization, Memory Management এবং Web Application Efficiency

Practical Project: DataPulse Explorer — ১০,০০০ রেকর্ডের Search, Sort ও Virtual Scrolling Dashboard।

Learning Outcomes

এই অধ্যায় শেষে তুমি পারবে (১) Performance Bottleneck চিহ্নিত করতে, (২) Time Complexity ও বাস্তব Benchmark-এর পার্থক্য বুঝতে, (৩) Browser Main Thread-এর কাজ কমাতে, (৪) Memory Leak প্রতিরোধ করতে, (৫) Virtualization ও Debounce প্রয়োগ করতে, এবং (৬) DevTools দিয়ে পরিবর্তনের ফল যাচাই করতে।

১৯.১ Performance বলতে কী বোঝায়?

একটি Web Application দ্রুত মনে হওয়ার জন্য শুধু JavaScript Function দ্রুত হওয়া যথেষ্ট নয়। Network, JavaScript Execution, Style Calculation, Layout, Paint এবং User Input Response—সবকিছু মিলে ব্যবহারকারীর অভিজ্ঞতা তৈরি হয়। **Latency** হলো একটি কাজ শেষ হতে সময়; **Throughput** হলো নির্দিষ্ট সময়ে কত কাজ করা যায়। ১০,০০০ রেকর্ড Filter করতে ৩ ms লাগলেও ১০,০০০ DOM Row আঁকতে ২০০ ms লাগতে পারে। তাই Optimization-এর আগে কোথায় সময় ব্যয় হচ্ছে তা জানতে হবে।

Rule: Measure → Identify Bottleneck → Change One Thing → Measure Again। অকারণে `for` বনাম `map` নিয়ে Micro-optimization করলে বড় Bottleneck অমীমাংসিত থাকতে পারে।

১৯.২ Big-O: Algorithmic Growth

Complexity	সাধারণ উদাহরণ	১০ থেকে ১০,০০০ Input হলে
O(1)	`Map.get(key)` — average-case	তুলনামূলক স্থির
O(log n)	Sorted Data-তে Binary Search	ধীরে বাড়ে
O(n)	Array Scan	প্রায় Input-এর অনুপাতে
O(n log n)	Efficient Comparison Sort	n-এর চেয়ে দ্রুত বাড়ে
$O(n^2)$	Nested Pairwise Comparison	বড় Data-তে অত্যন্ত ব্যয়বহুল

```javascript
// O(n²): প্রতিটি Product-এর সঙ্গে সব Product মিলানো
for (const a of products) {
  for (const b of products) {
    if (a.id === b.id) { /* ... */ }
  }
}

// ID Lookup বারবার দরকার হলে একটি Map তৈরি করা ভালো।
const byId = new Map(products.map(item => [item.id, item]));
const product = byId.get(42); // average-case O(1)
```

`Map` তৈরিতে O(n) সময় ও অতিরিক্ত Memory লাগে। তাই একবার Lookup-এর জন্য সবসময় `Map` বানানো লাভজনক নয়। **Time–Space Tradeoff** বুঝে সিদ্ধান্ত নিতে হবে।

১৯.৩ Performance Measurement

```javascript
const start = performance.now();
const result = expensiveCalculation();
const duration = performance.now() - start;
console.log({ duration, result });
```

`performance.now()` High-Resolution Clock দেয়। কিন্তু একটি Measurement যথেষ্ট নয়: JIT Warm-up, Garbage Collection, Background Tab Throttling ও Machine Load-এর কারণে ফল বদলায়। একই Data দিয়ে বহুবার চালিয়ে Median/Distribution দেখো। Production User Experience যাচাই করতে Browser Performance Panel ও Real-User Metrics ব্যবহার করো।

```javascript
function benchmark(fn, iterations = 30) {
  for (let i = 0; i < 5; i++) fn(); // warm-up
  const samples = [];
  for (let i = 0; i < iterations; i++) {
    const t = performance.now();
    fn();
    samples.push(performance.now() - t);
  }
  samples.sort((a, b) => a - b);
  return { medianMs: samples[Math.floor(samples.length / 2)], samples };
}
```

সতর্কতা: Benchmark-এ Result ব্যবহার না করলে Optimizer কাজ বাদ দিতে পারে; Browser Rendering Benchmark-এ DOM Paint-এর খরচ আলাদা বিবেচনা করতে হবে।

১৯.৪ Main Thread ও Long Tasks

Browser-এর Main Thread JavaScript, Input Processing এবং Rendering-এর বহু কাজ সামলায়। দীর্ঘ Synchronous Loop User Interaction আটকে দিতে পারে।

```javascript
// বড় কাজকে Chunk-এ ভাগ করা: Browser-কে মাঝে কাজের সুযোগ দেওয়া।
async function processInChunks(items, process, chunkSize = 500) {
  for (let i = 0; i < items.length; i += chunkSize) {
    for (const item of items.slice(i, i + chunkSize)) process(item);
    await new Promise(resolve => setTimeout(resolve, 0));
  }
}
```

`setTimeout(0)` সঙ্গে সঙ্গে চলার নিশ্চয়তা দেয় না; এটি শুধু পরবর্তী Task-এ কাজ সরায়। Heavy CPU Computation-এর জন্য **Web Worker** ব্যবহার করা যায়; Worker সরাসরি DOM পরিবর্তন করতে পারে না। Large Data Transfer-এ Structured Clone ও Transferable Objects বিবেচনা করতে হয়।

১৯.৫ DOM Rendering কেন ধীর হয়?

```javascript
// সমস্যা: প্রতিবার ১০,০০০ Row DOM-এ রাখা
for (const item of items) {
  list.append(createRow(item));
}
```

অনেক DOM Node Memory ও Layout Cost বাড়ায়। `DocumentFragment` দিয়ে Batch Append কিছু Overhead কমাতে পারে, কিন্তু DOM-এ ১০,০০০ Row থাকলে সমস্যার মূল সমাধান হয় না। **Virtualization** শুধু দৃশ্যমান Row এবং সামান্য Overscan Render করে।

```javascript
const rowHeight = 40;
const start = Math.max(0, Math.floor(scrollTop / rowHeight) - 5);
const end = Math.min(total, Math.ceil((scrollTop + viewportHeight) / rowHeight) + 5);
const visibleItems = items.slice(start, end);
```

এই কৌশলটি Fixed-Height Rows ধরে। Variable-Height Rows হলে Measurement Cache ও Dynamic Positioning দরকার। Accessibility, Keyboard Navigation ও Screen Reader Support-ও পরীক্ষা করতে হবে।

১৯.৬ Debounce ও Throttle

User প্রতিটি Key চাপলে ১০,০০০ Item Search চালানো অপ্রয়োজনীয় হতে পারে। **Debounce** শেষ Input-এর পর নির্দিষ্ট সময় অপেক্ষা করে; **Throttle** নির্দিষ্ট Interval-এ সর্বোচ্চ একবার কাজ চালায়।

```javascript
function debounce(fn, delay = 200) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
searchInput.addEventListener('input', debounce(updateResults, 180));
```

Debounce-এর ফলে Response একটু দেরিতে আসে; Input Type ও UX অনুযায়ী Delay বেছে নিতে হবে। Scroll Rendering-এ `requestAnimationFrame` সাধারণত উপযোগী।

১৯.৭ Memory Model ও Garbage Collection

JavaScript Engine Reachability অনুসারে অপ্রয়োজনীয় Object-এর Memory Reclaim করে। `const` / `let` ব্যবহার করলেই Memory Leak বন্ধ হয় না। **Leak** হলো প্রয়োজন শেষ হওয়ার পরও কোনো Object Reference ধরে রাখা।

```javascript
const handler = () => console.log('resize');
window.addEventListener('resize', handler);

// Component বা Feature বন্ধ করার সময়:
window.removeEventListener('resize', handler);
```

সম্ভাব্য Leak Source: Unremoved Event Listener, `setInterval`, Unbounded Cache, Detached DOM Nodes, Global Collections, Long-Lived Closure। `WeakMap`-এ Object Key-এর Weak Reference থাকে, তবে এটি সব Leak-এর সমাধান নয়। Memory Tab-এ Heap Snapshot ও Retainer Paths দেখো।

১৯.৮ Cache: সুবিধা ও ঝুঁকি

Repeated Expensive Computation-এর জন্য Memoization উপকারী, তবে Cache Memory নেয় এবং Stale Data-এর ঝুঁকি থাকে।

```javascript
const cache = new Map();
function cachedSummary(key, items) {
  if (cache.has(key)) return cache.get(key);
  const value = items.reduce((sum, x) => sum + x.price, 0);
  cache.set(key, value);
  return value;
}
```

এখানে `key` যদি Data Version অনুযায়ী না বদলায়, ভুল Result পাওয়া যাবে। Unbounded `Map` সময়ের সঙ্গে বড় হতে পারে। TTL, LRU, Max Size অথবা Explicit Invalidation প্রয়োজন হতে পারে।

১৯.৯ Network, Bundles ও Lazy Loading

সব Performance সমস্যা JavaScript Loop-এর নয়। বড় Bundle, Blocking Script, Large Images, অতিরিক্ত API Calls এবং Third-Party Scripts Page Slow করতে পারে। `defer`, ES Modules, Dynamic `import()`, Code Splitting, Compression, Cache Headers ও Image Optimization প্রাসঙ্গিক।

```javascript
// ব্যবহারকারী Feature খুললেই Heavy Module Load
button.addEventListener('click', async () => {
  const { renderChart } = await import('./charts.js');
  renderChart();
});
```

`import()` Promise Return করে; Loading Error Handle করতে হবে। Browser DevTools Network Panel-এ Download Size, Waterfall ও Cache Behavior যাচাই করো।

১৯.১০ Practical Project — DataPulse Explorer

Scenario: একটি Dashboard-এ ১০,০০০ Product-like Record আছে। Search, Category Filter ও Sorting থাকবে। কিন্তু Screen-এ একসঙ্গে হাজার হাজার Row না এঁকে Virtual Window ব্যবহার করব।

Project Structure

```
source-code/chapter-19/
├── index.html
├── style.css
├── data-core.mjs
├── app.mjs
└── data-core.test.mjs
```

Run: Project Directory-তে `python -m http.server 8000` চালিয়ে `http://localhost:8000` খুলো। ES Modules `file://` থেকে চালালে Browser Security Restrictions দেখা দিতে পারে। Tests: `node --test data-core.test.mjs`।

Domain Logic: `createDataset()` Deterministic Records তৈরি করে; `queryDataset()` Filter ও Sort করে, Original Array Mutate করে না; `summarize()` Count ও Average হিসাব করে; `windowRange()` Visible Slice-এর Index দেয়। Source Files-এ সম্পূর্ণ Implementation আছে।

```javascript
const all = createDataset(10000);
const result = queryDataset(all, { category: 'Books', sort: 'price-desc' });
const { start, end } = windowRange(result.length, 1200, 40, 400, 6);
const visible = result.slice(start, end);
```

UI Rendering: `app.mjs`-এ Scroll Event-কে `requestAnimationFrame` দিয়ে Batch করা হয়েছে; DOM-এ শুধু Visible Rows রাখা হয়। `textContent` দিয়ে Safe Text Rendering করা হয়েছে। Search Input Debounced; Filter ও Sort Change সঙ্গে সঙ্গে Apply হয়। Query Duration `performance.now()` দিয়ে মাপা হয়।

Performance Experiment: DevTools Elements Panel-এ দেখো ১০,০০০ Record থাকা সত্ত্বেও Rendered Row সাধারণত কয়েক ডজন। তারপর Search ও Scroll করে FPS/Responsiveness পর্যবেক্ষণ করো। এটি Virtualization-এর সুফল দেখায়; সব Device-এ নির্দিষ্ট Millisecond Performance-এর নিশ্চয়তা নয়।

১৯.১১ Debugging & Profiling Checklist

1. Performance Panel-এ Record চালু করে Search/Scroll করো।
2. Main Thread-এ Long Task আছে কি না দেখো।
3. Scripting, Rendering ও Painting Time আলাদা করো।
4. Memory Heap Snapshot নিয়ে Repeat Interaction-এর পরে Compare করো।
5. Filter/Sort Result Correct কি না Automated Tests চালাও।
6. Low-End Device ও Slow Network Emulation-এ Test করো।
7. Keyboard, Focus ও Screen Reader Usability যাচাই করো।

১৯.১২ Common Mistakes

- **Premature Optimization:** Problem না মেপে Code জটিল করা।
- `sort()` **Mutation:** Original Array বদলে যায়; Copy করে Sort করো।
- **Unbounded Cache:** Cached Object কখনো Clear না করা।
- **Scroll Handler Overwork:** প্রতিটি Scroll Event-এ Heavy Computation চালানো।
- **Virtualization = Free Performance:** Filtering/Sorting এখনো সম্পূর্ণ Array-এ চলে।
- **Benchmark Confusion:** One-off Local Timing-কে Production Performance ধরে নেওয়া।
- **Detached DOM Leak:** Listener/Reference Cleanup না করা।

১৯.১৩ Exercises — ১০টি

1. `performance.now()` দিয়ে Array Filter-এর Duration মাপো।
2. ১০,০০০ Record থেকে `category === 'Books'` Filter করো।
3. Original Array অপরিবর্তিত রেখে Price Descending Sort করো।
4. Empty Array-এর Average শূন্য Return করার Function লেখো।
5. ৪০ px Row Height ও ৪০০ px Viewport-এর Visible Range বের করো।
6. Debounce Function লেখো এবং Rapid Calls-এর Result পরীক্ষা করো।
7. `Map`-ভিত্তিক ID Lookup তৈরি করো।
8. `setInterval`-এর Cleanup Function লেখো।
9. ৫০০টি Item করে Async Chunk Processing Implement করো।
10. DataPulse-এর Filter/Sort ও Virtualization Test Cases যোগ করো।

১৯.১৪ Sample Solutions

JAVASCRIPT

```
// 1
const t0 = performance.now();
const expensiveResult = numbers.filter(n => n % 2 === 0);
console.log(performance.now() - t0);
```

```
// 2
const books = items.filter(item => item.category === 'Books');

// 3
const descending = [...items].sort((a, b) => b.price - a.price);

// 4
const average = xs => xs.length ? xs.reduce((s, x) => s + x, 0) / xs.length : 0;

// 5
const start = Math.max(0, Math.floor(scrollTop / 40) - 5);
const end = Math.min(totalRows, Math.ceil((scrollTop + 400) / 40) + 5);

// 6: source-code/chapter-19/data-core.mjs → debounce()

// 7
const byId = new Map(items.map(item => [item.id, item]));
console.log(byId.get(42));

// 8
const interval = setInterval(refresh, 1000);
function cleanup() { clearInterval(interval); }

// 9
async function chunk(items, fn) {
  for (let i = 0; i < items.length; i += 500) {
    items.slice(i, i + 500).forEach(fn);
    await new Promise(resolve => setTimeout(resolve, 0));
  }
}

// 10: source-code/chapter-19/data-core.test.mjs দেখো।
```

Exercise Note: `numbers`, `items`, `scrollTop`, `totalRows`, `refresh` এগুলো নিজস্ব Example Data/Function দিয়ে Define করতে হবে।

১৯.১৫ Knowledge Check — ১০টি MCQ

১. Virtualization-এর মূল সুবিধা কী? A) API Faster B) DOM Nodes কমানো C) Database Encryption D) Type Checking

২. `Array.sort()` কী করে? A) Always New Array B) Original Mutate C) Network Request D) Promise Return

৩. Debounce কখন উপকারী? A) Rapid Search Input B) Password Hashing C) CSS Color D) Database Migration

৪. `performance.now()` কী মাপে? A) Duration Timing B) Disk Size C) IP Address D) Heap Object Count

৫. Memory Leak-এর একটি কারণ? A) Removed Listener B) Unbounded Cache C) Local Constant D) Small Function

৬. Web Worker কি DOM সরাসরি পরিবর্তন করতে পারে? A) হ্যাঁ B) না C) শুধু Chrome-এ D) শুধু CSS-এ

৭. `Map.get()` Average-case Lookup? A) $O(n^2)$ B) O(log n) C) O(1) D) O(n!)

৮. `requestAnimationFrame` সাধারণত কোথায় উপকারী? A) Visual Update B) SQL Query C) File Compression

D) Password Storage

৯. `WeakMap` Key হিসেবে কী নেয়? A) Arbitrary String B) Objects এবং Non-registered Symbols C) Numbers only D) Arrays only

১০. Optimization-এর প্রথম ধাপ কী? A) Guess B) Measure C) Rewrite Everything D) Add More Libraries

Answers & Explanations

1. **B** — দৃশ্যমান Row-এর বাইরে DOM তৈরি না করলে Rendering Cost কমে।
2. **B** — `sort()` In-place Mutate করে।
3. **A** — দ্রুত Input Events Coalesce করা যায়।
4. **A** — High-Resolution Timestamp দিয়ে Elapsed Time মাপা হয়।
5. **B** — Unlimited Cache Reference ধরে রাখে।
6. **B** — Worker-এর সরাসরি DOM Access নেই।
7. **C** — সাধারণ Hash Map Implementation-এ Average-case Constant Time।
8. **A** — Browser Paint Cycle-এর সঙ্গে Visual Work Schedule করে।
9. **B** — Object এবং Non-registered Symbol Key Supported।
10. **B** — Measurement ছাড়া Bottleneck নির্ধারণ করা যায় না।

১৯.১৬ Chapter Recap ও Next Step

Performance Engineering হলো **Measured Tradeoff**। Algorithm, DOM, Main Thread, Memory ও Network একসঙ্গে দেখতে হয়। DataPulse Explorer-এ Pure Data Queries, Debounce, Virtual Window এবং Profiling-এর ভিত্তি তৈরি হলো। পরবর্তী Chapter 20-এ Web Application Security ও Secure Coding Practices শিখব।